

Bayesian Matrix Factorization

SRIROTH Poonyapat 1155205059

YANG Yanze 1155210986

Department of Computer Science and Engineering

April 30, 2025

1 Introduction and Problem Statement

Matrix completion is a fundamental problem in recommender systems where we are given a sparse rating matrix $\mathbf{R}$ of size $N \times M$, representing ratings from N users on M items. Most entries in this matrix are unknown, with only L observed ratings ($L \ll N \times M$). For example, in movie recommendation systems, users rate only a small subset of available movies. The goal is to predict all unrated items by estimating approximately $N \times M - L \approx N \times M$ parameters.

Predicting each missing entry independently would require an impractical number of parameters. Matrix factorization provides a dimension reduction approach by approximating the rating matrix $\mathbf{R}$ as a product of two lower-dimensional matrices: $\mathbf{R} \approx \mathbf{U}\mathbf{V}^T$, where $\mathbf{U}$ is an $N \times K$ matrix representing user factors and $\mathbf{V}$ is an $M \times K$ matrix representing item factors, with $K \ll \min(N, M)$. Each rating is approximated as $r_{ij} \approx \mathbf{u}_i \cdot \mathbf{v}_j^T$. This reduces the parameter space from $N \times M$ to $(N + M) \times K$. The objective function typically minimizes the squared error over observed ratings: $\min_{\mathbf{U}, \mathbf{V}} \sum_{(i,j) \in \mathcal{O}} (r_{ij} - \mathbf{u}_i \cdot \mathbf{v}_j^T)^2$, where $\mathcal{O}$ denotes the set of observed ratings.

2 Bayesian Matrix Factorization

2.1 Solving Bayesian Matrix Factorization

2.1.1 Prior Distributions

In BMF, each row of matrices $\mathbf{U}$ and $\mathbf{V}$ represents the latent factors for users and items, respectively, as a column vector. We place standard normal priors on these factors:

$$\begin{cases} u_{ik} \sim \mathcal{N}(0, 1) \\ v_{jk} \sim \mathcal{N}(0, 1) \end{cases} \quad \text{for any } 1 \leq k \leq K \quad (1)$$

The probability density functions for the user and item factor vectors are:

$$f(\mathbf{u}_i) = \frac{1}{(2\pi)^{K/2}} \exp\left(-\frac{1}{2} \mathbf{u}_i^T \mathbf{u}_i\right) \quad (2)$$

$$f(\mathbf{v}_j) = \frac{1}{(2\pi)^{K/2}} \exp\left(-\frac{1}{2} \mathbf{v}_j^T \mathbf{v}_j\right) \quad (3)$$

2.1.2 Likelihood Function

For individual observed ratings, we model the likelihood as a normal distribution:

$$f(r_{ij}|\mathbf{u}_i, \mathbf{v}_j) = \mathcal{N}(r_{ij}|\text{sigmoid}(\mathbf{u}_i^T \mathbf{v}_j), \sigma^2) \quad (4)$$

Where the sigmoid function maps the dot product to a value in $[0, 1]$. This requires normalizing the original ratings:

$$r_{ij} = \frac{r_{ij} - 1}{R - 1} \in [0, 1] \quad (5)$$

where the original ratings are in the set $\{1, 2, \dots, R\}$, and σ^2 is a noise variance hyper-parameter.

The likelihood of all observed ratings can then be expressed as:

$$f(\{r_{ij}\}|\mathbf{U}, \mathbf{V}) = \prod_{i=1}^N \prod_{j=1}^M (\mathcal{N}(r_{ij}|\text{sigmoid}(\mathbf{u}_i^T \mathbf{v}_j), \sigma^2))^{I_{ij}} \quad (6)$$

where I_{ij} is an indicator function equal to 1 if user i rated item j , and 0 otherwise.

2.1.3 Posterior Inference

Using Bayes' rule, we can derive the posterior distribution of the latent factors:

$$f(\mathbf{U}, \mathbf{V}|\{r_{ij}\}) \propto f(\{r_{ij}\}|\mathbf{U}, \mathbf{V}) \cdot f(\mathbf{U}) \cdot f(\mathbf{V}) \quad (7)$$

$$\propto f(\{r_{ij}\}|\mathbf{U}, \mathbf{V}) \cdot \prod_{i=1}^N \frac{1}{(2\pi)^{K/2}} \exp\left(-\frac{1}{2}\mathbf{u}_i^T \mathbf{u}_i\right) \cdot \prod_{j=1}^M \frac{1}{(2\pi)^{K/2}} \exp\left(-\frac{1}{2}\mathbf{v}_j^T \mathbf{v}_j\right) \quad (8)$$

$$\propto \prod_{i=1}^N \prod_{j=1}^M (\mathcal{N}(r_{ij}|\text{sigmoid}(\mathbf{u}_i^T \mathbf{v}_j), \sigma^2))^{I_{ij}} \cdot \exp\left(-\frac{1}{2} \sum_{i=1}^N \mathbf{u}_i^T \mathbf{u}_i\right) \cdot \exp\left(-\frac{1}{2} \sum_{j=1}^M \mathbf{v}_j^T \mathbf{v}_j\right) \quad (9)$$

2.2 Making Prediction

After obtaining the posterior distribution of the latent factors, we can make predictions for unobserved ratings. For an unobserved rating r_{ab} between user a and item b , the predictive distribution is given by:

$$f(r_{ab}|\{r_{ij}\}) = \int f(r_{ab}|\mathbf{u}_a, \mathbf{v}_b) f(\mathbf{u}_a, \mathbf{v}_b|\{r_{ij}\}) d\mathbf{u}_a d\mathbf{v}_b \quad (10)$$

$$= \mathbb{E}_{\mathbf{u}, \mathbf{v} \sim f(\mathbf{u}, \mathbf{v}|\{r_{ij}\})} [f(r_{ab}|\mathbf{u}_a, \mathbf{v}_b)] \quad (11)$$

Maximum A Posteriori (MAP) Estimation In general, we can use Maximum A Posteriori (MAP) estimation to generate a predicted rating in $[0, 1]$:

$$r^* = \arg \max_r f(r_{ab}|\{r_{ij}\}) \quad (12)$$

However, the challenge in implementing this approach is that finding the exact mode of $f(r_{ab}|\{r_{ij}\})$ is computationally difficult.

Practical Implementation for Discrete Ratings To address that challenge, we can use the fact that ratings in a real system are discrete rather than continuous by:

1. Evaluating the probability density function over the set of R discrete normalized ratings:

$$\left\{ \frac{0}{R-1}, \frac{1}{R-1}, \frac{2}{R-1}, \dots, \frac{R-1}{R-1} \right\} \quad (13)$$

2. Selecting the rating with the highest probability as our prediction r^* .
3. Converting the normalized prediction back to the original rating scale:

$$r = (R-1)r^* + 1 \quad (14)$$

This transforms our prediction from the $[0, 1]$ interval back to the original rating scale $\{1, 2, \dots, R\}$.

2.3 Computational Challenges

While Bayesian Matrix Factorization offers theoretical advantages, it introduces significant computational challenges:

$$f(\mathbf{U}, \mathbf{V}|\{r_{ij}\}) = \frac{f(\mathbf{U})f(\mathbf{V})f(\{r_{ij}\}|\mathbf{U}, \mathbf{V})}{Z(\{r_{ij}\})} \quad (15)$$

$$f(r_{ab}|\{r_{ij}\}) = \int f(r_{ab}|\mathbf{u}_a, \mathbf{v}_b)f(\mathbf{u}_a, \mathbf{v}_b|\{r_{ij}\})d\mathbf{u}_a d\mathbf{v}_b \quad (16)$$

These equations present a major computational challenge as the posterior distribution cannot be calculated directly. Finding the posterior distribution requires computing the normalization constant $Z(\{r_{ij}\})$, which requires evaluating high-dimensional integrals such that, for a real-world situation, there might be over a million dimensions.

3 Sampling Approach

To address the computational challenges in Bayesian Matrix Factorization, we can use sampling-based methods to approximate the complex posterior and predictive distributions.

3.1 General Idea

Bayesian Inference Recap In the Bayesian framework, we have:

$$f_{\Theta|\mathcal{D}}(\theta|\mathcal{D}) = \frac{f_{\Theta}(\theta)f_{\mathcal{D}|\Theta}(\mathcal{D}|\theta)}{Z(\mathcal{D})} \quad (17)$$

where $Z(\mathcal{D}) = \int f_{\Theta}(\theta)f_{\mathcal{D}|\Theta}(\mathcal{D}|\theta)d\theta$ is the normalization constant. The predictive distribution for a new observation x^* is given by:

$$P(x^*|\mathcal{D}) = \int_{-\infty}^{+\infty} P(x^*|\theta) f_{\Theta|\mathcal{D}}(\theta|\mathcal{D}) d\theta \quad (18)$$

$$= \mathbb{E}_{\theta \sim f(\theta|\mathcal{D})} [P(x^*|\theta)] \quad (19)$$

Computing this predictive distribution requires integrating over the posterior, which is analytically intractable for BMF.

Sampling Solution we can approximate them using sampling:

1. Draw independent samples $\{\theta_1, \theta_2, \dots, \theta_n\}$ from the posterior distribution $f(\theta|\mathcal{D})$
2. Approximate the expectation using the sample mean:

$$\mathbb{E}_{\theta \sim f(\theta|\mathcal{D})} [P(x^*|\theta)] \approx \frac{1}{n} \sum_{i=1}^n P(x^*|\theta_i) = \hat{P}(x^*|\theta) \quad (20)$$

This sample mean estimator is unbiased:

$$\mathbb{E}_{\theta \sim f(\theta|\mathcal{D})} [P(x^*|\theta)] = \mathbb{E}_{\theta \sim f(\theta|\mathcal{D})} [\hat{P}(x^*|\theta)] \quad (21)$$

Problem Setup for Sampling In the context of BMF, we can formulate our problem as:

- $\mathbf{z}$ represents our model parameters $(\mathbf{U}, \mathbf{V})$ - a multivariate random variable
- We aim to evaluate the expectation:

$$\mathbb{E}_{\mathbf{z} \sim f(\mathbf{z})} [h(\mathbf{z})] = \int h(\mathbf{z}) f(\mathbf{z}) d\mathbf{z} \quad (22)$$

- The distribution $f(\mathbf{z}) = \frac{g(\mathbf{z})}{Z}$ where $g(\mathbf{z})$ can be evaluated pointwise (the unnormalized posterior), but Z cannot be directly computed (the intractable normalization constant)
- Our objective is to draw independent samples $\{\mathbf{z}_1, \dots, \mathbf{z}_n\}$ from $f(\mathbf{z})$ to approximate:

$$\mathbb{E}_{\mathbf{z} \sim f(\mathbf{z})} [h(\mathbf{z})] \approx \frac{1}{n} \sum_{i=1}^n h(\mathbf{z}_i) \quad (23)$$

In the BMF context, $g(\mathbf{z})$ corresponds to the product of the prior and likelihood:

$$g(\mathbf{U}, \mathbf{V}) = f(\mathbf{U}) f(\mathbf{V}) f(\{r_{ij}\}|\mathbf{U}, \mathbf{V}) \quad (24)$$

And $h(\mathbf{z})$ represents the predictive distribution for a specific unobserved rating:

$$h(\mathbf{U}, \mathbf{V}) = f(r_{ab}|\mathbf{u}_a, \mathbf{v}_b) \quad (25)$$

3.2 Markov Chain

A fundamental concept in sampling-based inference is the Markov chain. A sequence of random variables $\{\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_n\}$ forms a **first-order Markov chain** when the distribution of the next state depends only on the current state:

$$P(\mathbf{z}_{k+1}|\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_k) = P(\mathbf{z}_{k+1}|\mathbf{z}_k) \quad \text{for } k \in \{1, \dots, n-1\} \quad (26)$$

To fully specify a Markov chain, we need:

- An initial distribution $P(\mathbf{z}_1)$
- A transition kernel $T(\mathbf{z}_{k+1} \leftarrow \mathbf{z}_k) \equiv P(\mathbf{z}_{k+1}|\mathbf{z}_k)$

Stationary A distribution $P^*(\cdot)$ is **stationary** with respect to a Markov chain if:

$$P^*(\mathbf{z}) = \sum_{\mathbf{z}'} T(\mathbf{z} \leftarrow \mathbf{z}') P^*(\mathbf{z}') \quad (27)$$

Detailed Balance A sufficient condition for ensuring that $P^*(\cdot)$ is stationary is **detailed balance**:

$$T(\mathbf{z}' \leftarrow \mathbf{z}) P^*(\mathbf{z}) = T(\mathbf{z} \leftarrow \mathbf{z}') P^*(\mathbf{z}') \quad (28)$$

3.3 Markov Chain Monte Carlo

Markov Chain Monte Carlo (MCMC) is a powerful technique for sampling from complex probability distributions when direct sampling is infeasible.

MCMC Approach The core idea of MCMC is to:

- Construct a Markov chain whose stationary distribution is our target distribution $f(\mathbf{z})$
- Run the chain for a sufficiently long time until it reaches approximate stationarity
- Collect samples after a burn-in period

These samples will approximate draws from $f(\mathbf{z})$, even if the initial state was not drawn from the target distribution.

Practical Considerations Since consecutive samples in a Markov chain are correlated, we often discard an initial set of samples (burn-in period) to reduce the influence of the starting state, and keep only every T -th sample $\{\mathbf{z}_T, \mathbf{z}_{2T}, \dots, \mathbf{z}_{nT}\}$ to reduce the correlation between each sampling.

3.4 Metropolis-Hasting Algorithm

The Metropolis-Hastings (M-H) algorithm provides a method to sample from complex probability distributions without computing normalization constants. It constructs a Markov chain that converges to the target distribution.

Algorithm Description The M-H algorithm defines a transition from current state $\mathbf{z}$ to a new state $\mathbf{z}'$ through:

1. Generate a candidate state $\mathbf{z}^*$ from a proposal distribution $q(\mathbf{z}^*|\mathbf{z})$
2. Calculate the acceptance probability:

$$A(\mathbf{z}^*, \mathbf{z}) = \min \left(1, \frac{f(\mathbf{z}^*)q(\mathbf{z}|\mathbf{z}^*)}{f(\mathbf{z})q(\mathbf{z}^*|\mathbf{z})} \right) \quad (29)$$

$$= \min \left(1, \frac{g(\mathbf{z}^*)q(\mathbf{z}|\mathbf{z}^*)}{g(\mathbf{z})q(\mathbf{z}^*|\mathbf{z})} \right) \quad (30)$$

3. Set $\mathbf{z}' = \mathbf{z}^*$ with probability $A(\mathbf{z}^*, \mathbf{z})$; otherwise, set $\mathbf{z}' = \mathbf{z}$

Key Properties The M-H algorithm has two essential features:

- **Preference for high-probability regions:** When $f(\mathbf{z}^*) > f(\mathbf{z})$, the move to $\mathbf{z}^*$ is likely accepted, guiding the chain toward regions of high probability.
- **Proposal bias correction:** The ratio $\frac{q(\mathbf{z}|\mathbf{z}^*)}{q(\mathbf{z}^*|\mathbf{z})}$ adjusts for asymmetry in the proposal distribution.

The transition kernel $T(\mathbf{z}' \leftarrow \mathbf{z}) = q(\mathbf{z}'|\mathbf{z})A(\mathbf{z}', \mathbf{z})$ ensures that the chain satisfies detailed balance:

$$T(\mathbf{z}' \leftarrow \mathbf{z})f(\mathbf{z}) = T(\mathbf{z} \leftarrow \mathbf{z}')f(\mathbf{z}') \quad (31)$$

4 Implementation and Algorithms

4.1 Applying M-H Algorithm

For Bayesian Matrix Factorization, we implement the M-H algorithm to sample from the posterior distribution of the latent factors $\mathbf{U}$ and $\mathbf{V}$.

Sampling Procedure

1. **Initialization:** Initialize $\mathbf{U}$ and $\mathbf{V}$ randomly or using SVD (as it gives a better starting state)
2. **Proposal mechanism:** We use a Gaussian proposal distribution:

$$q(\mathbf{z}^*|\mathbf{z}) = \mathcal{N}(\mathbf{z}^*|\mathbf{z}, \sigma^2) \quad (32)$$

To generate candidates, we simply add Gaussian noise to the current parameters:

$$\mathbf{U}^* = \mathbf{U} + \epsilon_U, \quad \epsilon_U \sim \mathcal{N}(0, \sigma_U^2) \quad (33)$$

$$\mathbf{V}^* = \mathbf{V} + \epsilon_V, \quad \epsilon_V \sim \mathcal{N}(0, \sigma_V^2) \quad (34)$$

3. **Accept/Reject:** Compute the acceptance probability and decide whether to accept the next candidate by calculating the unnormalized posterior probability.

Prediction After collecting sufficient samples (discarding an initial burn-in period), we approximate the predictive distribution for an unobserved rating r_{ab} :

$$f(r_{ab}|\{r_{ij}\}) \approx \frac{1}{S} \sum_{s=1}^S f(r_{ab}|\mathbf{u}_a^{(s)}, \mathbf{v}_b^{(s)}) \quad (35)$$

$$= \frac{1}{S} \sum_{s=1}^S \mathcal{N}(r_{ab}|\text{sigmoid}((\mathbf{u}_a^{(s)})^T \mathbf{v}_b^{(s)}), \sigma^2) \quad (36)$$

where S is the number of samples and $(\mathbf{u}_a^{(s)}, \mathbf{v}_b^{(s)})$ is the s -th sample from the posterior.

4.2 Implementation Details

Our Metropolis-Hastings implementation for Bayesian Matrix Factorization uses the following configuration:

Dataset For our experiments, we utilized the MovieLens Latest Small dataset, which contains 100,836 ratings from 610 users across 9,742 movies. The ratings range from 1 to 5 stars.

Data Processing The MovieLens ratings (1-5 stars) were normalized to $[0,1]$, with an 80/20 train/test split.

Model Parameters

- **Proposal variances:** $\sigma_U^2 = \sigma_V^2 = 0.005$
- **Latent dimension:** $K = 8$
- **Observation noise:** $\sigma^2 = 0.01$

MCMC Sampling

- **Iterations:** 2,000 total
- **Burn-in:** First 400 samples discarded
- **Thinning:** Every 5th sample retained

Initialization We initialize $\mathbf{U}$ and $\mathbf{V}$ using SVD on the observed rating matrix with missing values filled with the global mean rating.

5 Result

Our implementation of Bayesian Matrix Factorization demonstrated consistent improvement in prediction quality as the number of MCMC iterations increased. Figure 1 shows the percentage of correct predictions over iterations, Figure 2 displays the Root Mean Square Error, and Figure 3 illustrates the trace of the log posterior.

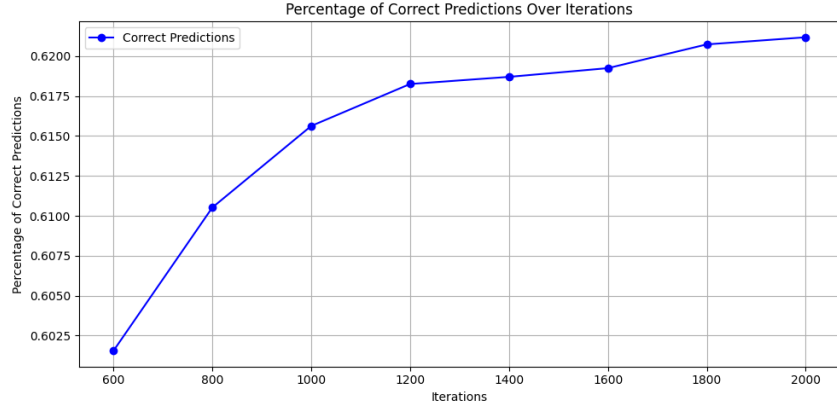


Figure 1: Percentage of Correct Predictions Over Iterations. The graph shows that, after the burn-in period and the Markov Chain is stationary distribution, the correctness of the prediction is approximately 60% and increases slightly over the iterations.

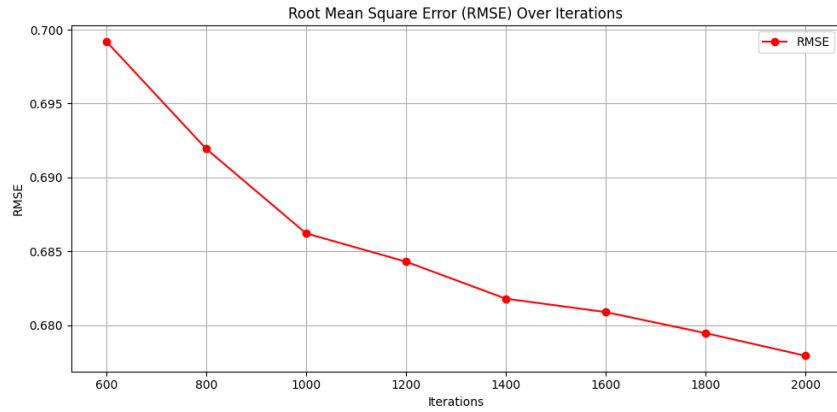


Figure 2: Root Mean Square Error (RMSE) Over Iterations. The RMSE decreases over the iterations, indicating improved prediction quality as more samples are collected.

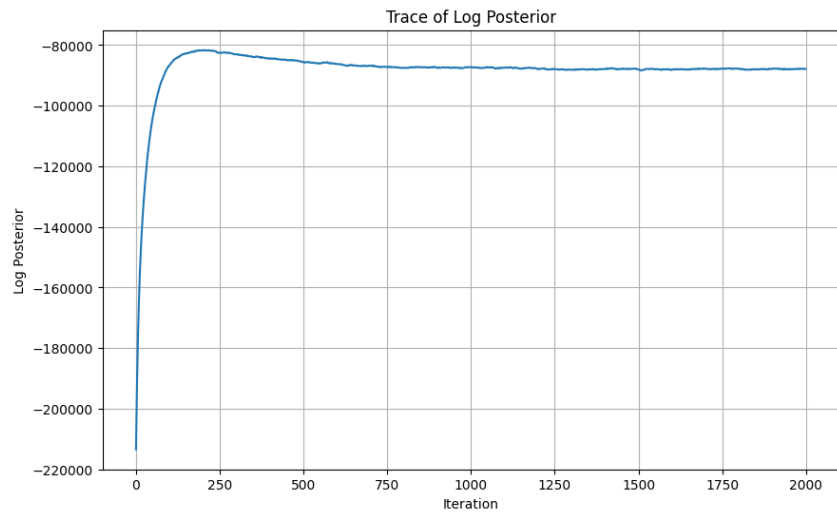


Figure 3: Trace of Log Posterior. The log posterior rapidly increases during the initial iterations (burn-in period) and then stabilizes after around 400-500 iterations, indicating convergence of the Markov chain. There is a slight decrease in the log posterior at around 200-400 iterations as a result of balancing the prior and likelihood of the posterior distribution.